ApTap — Reconstructed AI Engineering Dialogue

Enhanced edition | 15 October 2026 | Data engineering, cloud deployment and MLOps

> **SYNTHETIC ENGINEERING DESIGN RECONSTRUCTION — NOT AN ORIGINAL AI CHAT TRANSCRIPT.** This is a newly authored, structured reviewer–engineer discussion reconstructed from the project's documented decisions and recorded local test outputs. Its questions and responses are editorial, not historical quotations or a reconstruction of hidden reasoning. **Submit original complete AI conversations, share links and any coding-agent logs separately.**

Scope. A take-home prototype processes three monthly feeds for 1,500 synthetic customers; the employer can add an October JSONL delivery. Part D asks what was wrong with the data, what cannot be trusted, and how this would run in production. Two production locations are considered: ApTap-managed AWS (where an authorised bank export is permissible) and a fully bank-contained environment (where no customer information or inference leaves the bank).

Evidence vocabulary: [OBSERVED] input-file EDA or reported local execution; **[TESTED—RECORDED]** reported test output, not independently rerun for this revision; **[IMPLEMENTED—REPORTED]** behaviour recorded for the Release 2 local implementation; **[PROPOSED]** future production control or experiment; **[UNKNOWN]** contractual, operational or validation fact not established. All architecture drawings are proposed, not deployed.

01 — Start with the data, not the model

DESIGN REVIEWER — What did the initial exploratory data analysis reveal that actually changed the pipeline?

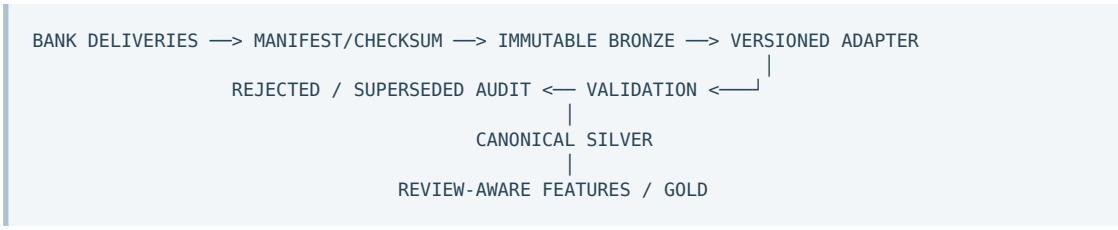
ENGINEER — The first question was whether the files represented the same business transaction schema. They did not. July and August use CSV with `transaction_id`, signed `amount`, explicit `currency` and ISO dates; September uses JSONL with `txn_id`, a `posted` day-first date, positive-magnitude `value`, `direction` and `narrative`. In September, 22,476 monetary values contain a `£` symbol. A simple file concatenation could change transaction signs or misparse dates. **[OBSERVED]**

The EDA also detected 1,052 July-booked transactions delivered in August. These are not automatically invalid: booking date and source-delivery month are separate facts. Forty July records lacked a `customer_id`, and 1,423 repeat-ID occurrences appeared across the feeds, including 312 September IDs with a changed payload. The 184 raw merchant strings reduce to 92 canonical descriptions after case normalisation. These findings motivate explicit adapters, identity guards, correction lineage and merchant-key normalisation rather than ad hoc fixes downstream. **[OBSERVED]**

Observed risk	Silver/ingestion response	Evidence/status
CSV → JSONL migration; renamed fields	Versioned source adapters; canonical required-column contract	[IMPLEMENTED—REPORTED]
`DD/MM/YYYY` vs `YYYY-MM-DD`; direction + GBP magnitude	Explicit dates, GBP parsing and sign from `direction`, never inferred from magnitude alone	[IMPLEMENTED—REPORTED]
Missing customer identity	Reject/quarantine with machine-readable reason; no invented customer linkage	[IMPLEMENTED—REPORTED]
Repeated or corrected transaction IDs	Bank-scoped stable key + content fingerprint + preserved supersession audit	[IMPLEMENTED—REPORTED]
Late July booking in August delivery	Separate `source_delivery_period` and `booking_month`; recompute affected Gold windows	[IMPLEMENTED—REPORTED]
Merchant wording variants	Keep raw counterparty text; derive normalised matching key	[IMPLEMENTED—REPORTED]

DESIGN REVIEWER — Where do data-quality gates go?

ENGINEER — A pre-ingestion delivery gate should validate checksum, manifest completeness, schema version, file readability and source row count before promotion from landing. Bronze preserves immutable source bytes and delivery metadata. The Bronze→Silver contract checks customer and transaction identity, allowed date formats, amount/direction/currency coherence, parseability and row-level rejects. Silver→Gold gates reconcile accepted and non-active/rejected records and verify feature provenance. Schema drift must fail visibly or use an explicitly approved new adapter. **[PROPOSED cloud gates; local validation and reconciliation IMPLEMENTED—REPORTED]**



DESIGN REVIEWER — How do we demonstrate reconciliation and replay integrity?

ENGINEER — The reported local run reconciles **135,155 source occurrences = 133,692 active clean + 1,463 rejected or superseded**. Four required outputs — `clean_transactions.csv`, `rejected_rows.csv`, `recurring_bills.csv` and `switches.csv` — were byte-identical on rerun; an identical manifest committed no new batches. The latest reported suite passed **54/54 tests**. Fresh three-month execution was **7.678 seconds** at **82.8 MiB peak RSS**, measured on one local machine, not a 15m-customer scalability claim. **[TESTED—RECORDED]**

02 — Corrections, identity and temporal truth

DESIGN REVIEWER — Why not simply deduplicate on customer, date, merchant and amount?

ENGINEER — A customer can make two identical debits on the same date. Deduplicating on those fields would erase genuine transactions. The bank reports `transaction_id` as stable through migration, so identity uses a **bank-scoped transaction ID**, while a canonical payload fingerprint distinguishes a duplicate resend from a changed record. The raw deliveries and non-active versions remain auditable. **[IMPLEMENTED—REPORTED]**

DESIGN REVIEWER — Which corrected record wins?

ENGINEER — The documented rule is **latest valid observed delivery**, with deterministic ordering and a tie-breaker. Invalid resends do not displace a valid active version. This is a reproducible operational convention, *not* authoritative bank truth: the feeds do not expose a reliable source version or correction timestamp. Production onboarding should request a monotonic source sequence, correction event or authoritative reconciliation endpoint; until supplied, the convention and resulting uncertainty are explicit. **[IMPLEMENTED—REPORTED / UNKNOWN source authority]**

DESIGN REVIEWER — What should an October reviewer test?

ENGINEER — Add fields, shuffle columns/JSON key order, vary money formatting and date strings, resend exact files, send a changed-payload duplicate, deliver July-booked transactions late, send an invalid correction and re-run the whole workflow. Confirm the four output schemas, no duplicate side effects, the audit reason trail, bounded window recomputation and advancing the reporting cut-off to **31 October 2026**. These are *proposed acceptance tests*, not claimed October test results. **[PROPOSED]**

03 — Why the classifier is deliberately selective

DESIGN REVIEWER — Why did we not train a conventional supervised classifier such as XGBoost?

ENGINEER — The bottleneck is verified examples, not computing power. We have **57 original labelled merchant descriptions** spread unevenly over **10 target categories**; only one gym and two home-insurance examples are present, and known labels for `OCTOPUS ENERGY` and `TESCO MOBILE` are inconsistent. More than 133,000 repeated transactions are not 133,000 independent merchant labels. A newly fitted XGBoost or neural classifier on 57 examples might learn naming artefacts, have no minority-class support in some validation folds, and give the appearance of certainty without true merchant/product evidence. We did not reject machine learning: the implemented approach combines **approved aliases** with character-level **TF-IDF retrieval**, while locally provisioned pretrained **MiniLM** acts as a diagnostic second opinion. **[OBSERVED + IMPLEMENTED —REPORTED]**

DESIGN REVIEWER — Explain the full decision rule. Does MiniLM vote?

ENGINEER — The production *interface* always returns one of the ten permitted category values. The *governance decision* is separate:

1. Normalise the merchant matching key without discarding original text.
2. If an approved, versioned alias applies, return its deterministic category and provider mapping under the currently configured acceptance policy.
3. Otherwise use character TF-IDF to propose the best lexical category.
4. Optionally record offline MiniLM semantic agreement/disagreement for diagnostics. **MiniLM has zero vote weight** in the returned category.
5. For an unresolved alias, output the provisional ten-class category **and** `needs_review=true`; do not treat that prediction as trusted bill/switch evidence.

[IMPLEMENTED—REPORTED] The method is more **auditable and conservative under scarce labels**, not empirically proved superior in top-1 accuracy to every possible ML model. Similarity scores are *not calibrated probabilities*, and two models agreeing does not supply ground truth.

DESIGN REVIEWER — What did the experiment actually establish?

Exploratory 57-label benchmark	Char TF-IDF	Offline MiniLM
Top-1 accuracy	**0.789**	0.754
Macro-F1	0.635	**0.657**
Macro average precision	**0.698**	0.654

ENGINEER — These are leave-one-description-out exploratory diagnostics, **not** an independently held-out merchant-family validation set. Macro-F1 equally weights categories and exposes weak minority-class performance; PR/AP complements it on imbalanced data. No model's score may be interpreted as production accepted-mapping precision. **[TESTED—RECORDED]**

DESIGN REVIEWER — Why do 21 unresolved descriptions affect 15,033 records?

ENGINEER — The model sees just **92 canonical descriptions** after normalisation, yet a few frequent descriptions recur across thousands of rows. **71 descriptions** are demo-accepted; **21** require review, affecting **15,033/133,692 = 11.24%** of active transactions. Thus **88.76% accepted coverage** means *operational acceptance under a demo policy*, not 88.76% verified correctness. Approving five additional merchants could lower review exposure only if a human has corroborated their category and provider identity. Accepting low-confidence predictions to inflate coverage would corrupt downstream features. **[OBSERVED + IMPLEMENTED—REPORTED]**

DESIGN REVIEWER — What remains an unresolved governance issue?

ENGINEER — The current accepted catalogue marks two generic Admiral insurance aliases as `CURATED_DEMO`, while a separate audit classifies them high risk (**851 transactions**). The audit currently *exposes*, but does not override, runtime acceptance; another high-risk description, AVIVA INSURANCE, remains unapproved. A **mandatory production hardening proposal** is a policy-override layer: `HIGH` risk without corroborating product evidence must go to review regardless of demo alias status. A shadow-policy exercise withheld the two Admiral aliases: recurring bill candidates changed from **6,681 to 6,401**, switches stayed **209**. This is a policy sensitivity calculation, not evidence that 280 bills were false. **[OBSERVED; enforcement PROPOSED]**

04 — Commercial outputs: what is and is not inferred

DESIGN REVIEWER — Are we detecting bills and genuine provider switches?

ENGINEER — We are detecting *candidates*. For bills, the pipeline groups trusted debit evidence by customer, provider and category, checks monthly continuity, and estimates next payment from history. **6,681 candidates** were recorded: **6,561** with recent consecutive-month evidence and **120** provisional; an independent validator checked **19,938 supporting transaction IDs**. The median individual debit is `typical_amount`, not necessarily the customer's total monthly outlay. **267** groups had repeated same-day debits and **seven** materially diverged from a monthly-total interpretation. **[TESTED—RECORDED]**

Switch detection yielded **209 candidate provider changes**, including **87 provisional** and **122** with repeated activity at the new provider; **zero** were externally confirmed. Of these candidates, **162** had a lower observed new payment and **47** a higher one. Among **154 energy** candidates, the mean old-minus-new difference was **GBP 14.17** (95% bootstrap interval **GBP 12.39–16.06**); mobile (**51**) had **GBP 2.69** (interval **−GBP 1.68–7.02**), and broadband (**4**) is too sparse for a strong conclusion. None proves formal cancellation, a comparable tariff, causal savings, annual benefit or ApTap attribution. **[TESTED—RECORDED]**

DESIGN REVIEWER — Why expose this uncertainty instead of hiding it in the executive dashboard?

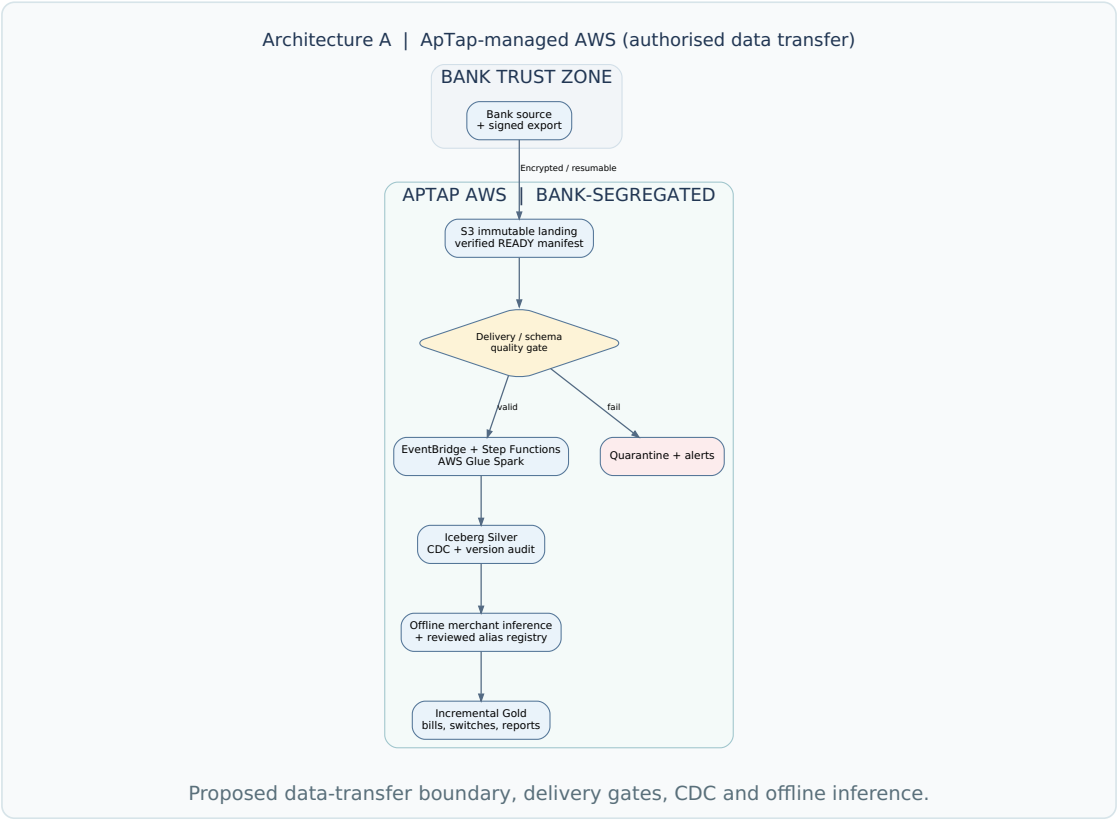
ENGINEER — A reporting system should separate *evidence*, *inference* and *commercial claim*. The one-page report is required; the supplemental offline risk dashboard visualises the raw→clean→review funnel, classifier disagreement and governance-policy sensitivity. Dashboard risk flags are diagnostic, **not runtime acceptance controls**. **[IMPLEMENTED—REPORTED]**

05 — Scenario A: ApTap-operated AWS, with authorised bank transfer

DESIGN REVIEWER — Assume the bank contract allows ApTap to process data in AWS. What is the target architecture?

ENGINEER — Place the trust boundary at the bank-approved transfer interface. Use a **signed/checksummed manifest**, resumable upload and an atomic `READY` marker only after all chunks and the whole object validate. ApTap receives only authorised fields into bank-segregated encrypted S3 landing. EventBridge invokes Step Functions; **AWS Glue Spark** is the default proposed processing engine, with EMR considered only after load testing establishes a reason. Validate source contracts and quarantine before table commits; use **Apache Iceberg** tables for canonical Silver and controlled `MERGE` with version audit; compute incremental customer/

period Gold; classify **unique merchant keys offline**, not each transaction repeatedly; publish outputs under role-limited policies. **[PROPOSED]**



Proposed data-transfer boundary, delivery gates, CDC and offline inference.

DESIGN REVIEWER — What makes this a production design rather than a cloud-service shopping list?

ENGINEER — Each layer owns a failure mode and an invariant. Landing/manifest owns **exactly-one complete delivery identity**; Step Functions owns checkpointed retries and abort conditions; Glue jobs own **schema/data contract validation** and row-level reconciliation; Iceberg owns atomic table commits, change-history and idempotent upsert semantics; Gold recomputes only affected booking windows after late corrections; classification owns review policy and provenance; reporting owns privacy restrictions and evidence disclaimers. These invariants must be integration tested before production. **[PROPOSED]**

Security is bank-specific data segregation, KMS encryption at rest, least-privilege IAM, VPC/private endpoints, defined retention/deletion, absence of raw PII in application logs, key rotation, incident alerts and restricted cross-bank joins. Billing/cost monitoring tracks **all-in cost per million records**, not just a model's inference cost. Local Python/SQLite does **not** prove Spark/Iceberg CDC parity. **[PROPOSED / UNKNOWN parity]**

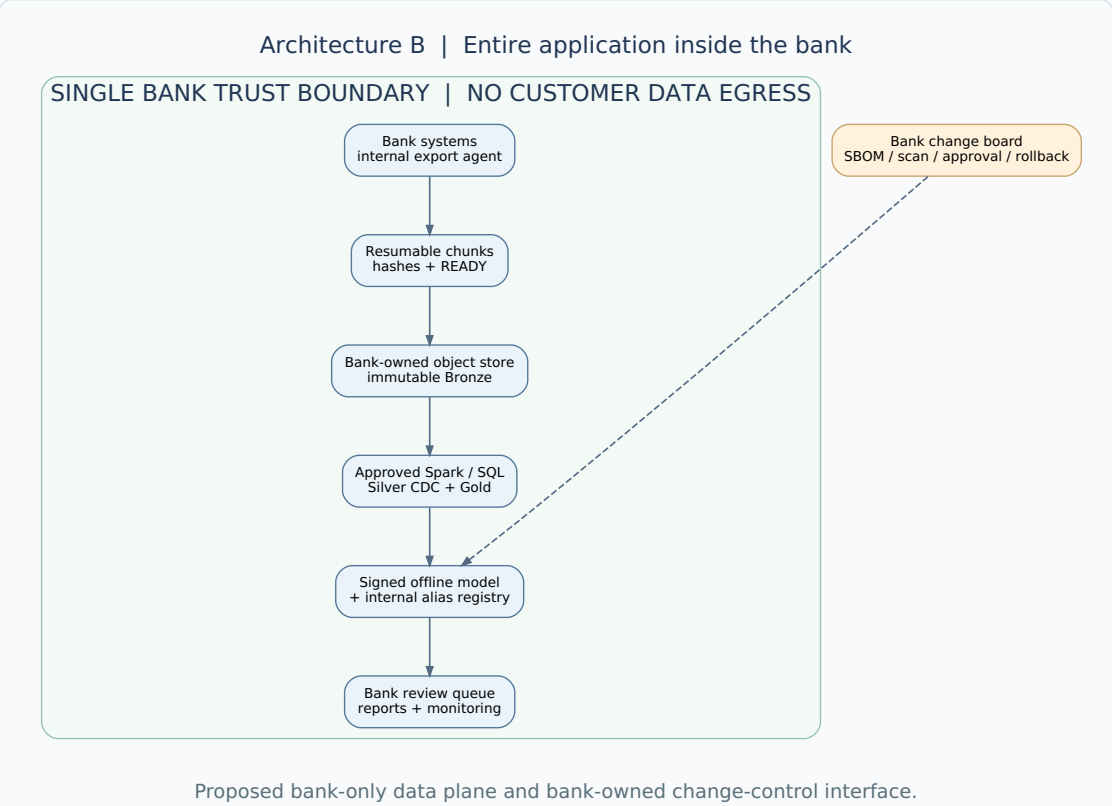
DESIGN REVIEWER — How is the classifier operated here?

ENGINEER — Pin the alias registry, lexical vocabulary, optional offline MiniLM artefact, adapter schema and code version into a reproducible deployment manifest. Infer once per **distinct normalised merchant identity** and distribute classifications via a controlled join. Human corrections create an approved registry revision. The MLOps evaluator uses reviewed examples, never exposes private descriptors to an external model API, and cannot auto-promote a candidate. **[PROPOSED]**

06 — Scenario B: the application lives wholly inside the bank

DESIGN REVIEWER — What changes if 15 million customers must be processed inside a bank where no customer data may leave and software installation is restricted?

ENGINEER — Keep the entire data plane and model execution inside the bank trust boundary. The bank uses an approved resumable transfer agent and internal object/file store; an atomic signed `READY` handoff prevents half-written loads. An approved native scheduler runs bank-approved Spark/SQL or reviewed containers against in-network Bronze/Silver/Gold; the bank controls encryption keys, review queue and reports. No public API calls, live model downloads, arbitrary `pip install`, customer-level telemetry export or automatic deployment from ApTap AWS. **[PROPOSED]**



Proposed bank-only data plane and bank-owned change-control interface.

DESIGN REVIEWER — How does a new model reach the bank?

ENGINEER — Through a **separate control-plane release**, not a data-plane export. ApTap can prepare a signed, pinned package with dependency hashes and software bill of materials; the bank scans it, tests it on synthetic or approved internal cases, signs off, and installs through its existing change-control system. The package could be a reviewed container, wheel bundle or native Spark/SQL alternative; its suitability depends on the bank's approved software catalogue. Any retraining requiring real bank data runs **inside the bank**, unless explicitly approved otherwise. The bank retains previous artefacts and can roll back atomically. **[PROPOSED; permitted packaging UNKNOWN]**

DESIGN REVIEWER — How do we handle unreliable massive uploads?

ENGINEER — Transfer chunked files with idempotent chunk IDs, per-chunk hashes, durable offset/checkpoint records, retries with bounded backoff and a verified whole-file checksum. Advance the manifest atomically to `READY` only after completeness and schema compatibility checks. Never infer that an upload is complete because its connection closed; never promote unverified partitions. **[PROPOSED]**

07 — Scale, performance and cost realism

DESIGN REVIEWER — What does the 15m-customer requirement imply quantitatively?

ENGINEER — September had **44,950 rows** for **1,500 customers**. Assuming unchanged transaction frequency, 15m customers implies **449.5 million rows/month**. Estimated September-density raw JSONL would be around **64.6 GB/month**, excluding indices, change-history, replication, temporary files and reports. This is an *extrapolation*, not a measured enterprise workload. **[PROPOSED assumption]**

The fresh laptop throughput, if naively scaled linearly, implies roughly **7.1 hours** for 449.5m rows, but that figure is expressly **not** a cloud SLA: SQLite single-writer contention, transaction-key lookup growth, wide joins, skew, file sizes, network transfer and history recomputations break linearity. **[OBSERVED reference + PROPOSED extrapolation]**

DESIGN REVIEWER — What will be partitioned and indexed?

ENGINEER — Use low-to-moderate-cardinality physical partitioning by **bank and booking month**, while retaining source delivery identifier and delivery month for lineage. Cluster/sort by customer and date in the selected table engine; maintain efficient equality lookup or a change-index for `(bank_id, transaction_id)`; compact small

files. A correction invalidates only its impacted customer/provider/month features and downstream reports. Do not physically partition by individual customer. Benchmark merge hotspots, late-arrival windows and top-merchant skew rather than assuming perfect distribution. **[PROPOSED]**

DESIGN REVIEWER — Can we estimate price per million transactions today?

ENGINEER — Only as a sensitivity illustration: **16 Glue DPUs × 4 h × an assumed USD 0.44/DPU-hour = USD 28.16**, or **about USD 0.063 per million** at 449.5m rows. This is a *hypothetical job-compute-only input*, neither verified provider pricing nor benchmarked throughput; it excludes S3, transfers, orchestration, security, drift review, audit, ML inference and support. The first acceptance test must measure **end-to-end all-in cost per million**, throughput, lag, failure/retry frequency and 95th-percentile batch latency at a realistic load. **[PROPOSED]**

08 — MLOps design: a measurable monthly operating policy

DESIGN REVIEWER — Which score should own the model-quality decision when categories are imbalanced?

ENGINEER — Macro-F1 is the proposed primary labelled metric, because each of the 10 categories contributes equally. Monitor one-vs-rest **average precision / precision–recall curves**, per-category precision and recall, particularly for insurance and sparse classes. Also monitor **selective quality**: accuracy/precision on the accepted cohort versus reviewed cohort, plus acceptance coverage and review cost. Never use *agreement*, TF-IDF similarity, or accepted share as substitutes for verified accuracy. **[PROPOSED]**

$$\text{Macro-F1} = \frac{1}{10}\sum_{c=1}^{10}\frac{2P_cR_c}{P_c+R_c} \qquad \text{Coverage} = \frac{N_{accepted}}{N_{clean}}$$

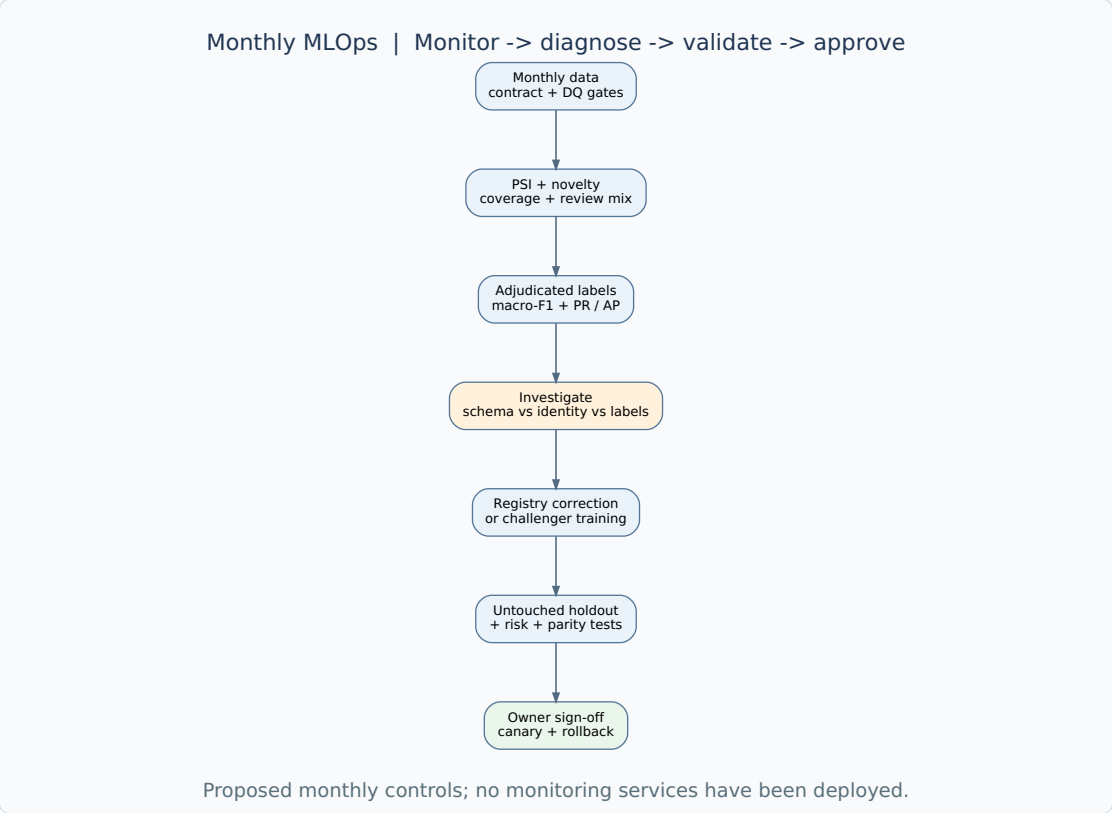
DESIGN REVIEWER — What is the naive benchmark, and why can’t 0.635 automatically become the production baseline?

ENGINEER — Compare a **majority-class constant predictor**, an **exact-approved-alias rules baseline** (report coverage and abstention separately), the frozen **TF-IDF champion**, and **MiniLM challenger** under the *same* held-out evaluation protocol. The existing **0.635 macro-F1** comes from exploratory leave-one-description-out on 57 labels, where a class with one example disappears from training when held out. It is not a reliable production

threshold. Create an **independently adjudicated, versioned, merchant-family-grouped holdout**, stratify for underrepresented and novel merchants, and use a rolling multi-month pool with uncertainty intervals. If samples are inadequate, mark performance `INSUFFICIENT_LABELS`, not falsely `HEALTHY`. **[PROPOSED]**

DESIGN REVIEWER — What is the monthly monitoring loop?

ENGINEER — On each completed monthly delivery, the data gate first checks freshness, manifest completeness, schema-version mix, parse failures, correction rate and raw→clean reconciliation. Next, detect distribution shifts in **merchant novelty rate, known/unknown share, text length, amount bands, category mix and review exposure**. Finally, evaluate macro-F1 and category precision/recall only on newly **independently adjudicated** descriptions/merchant groups. Log counts and confidence intervals, separately by bank and by month; compare to approved bank-specific/seasonally comparable reference windows. A PSI warning is not proof of concept drift. **[PROPOSED]**



Proposed monthly controls; no monitoring services have been deployed.

DESIGN REVIEWER — Make the alert thresholds explicit.

ENGINEER — After a proper labelled champion baseline is established, **investigate a 10% relative macro-F1 drop** (`current <= 0.90 × validated_baseline`). For example, **0.70 → 0.63** is a 10% relative decrease; 0.635 from the original exploratory benchmark is *not* substituted for the validated baseline. Require sufficient class support and either replication across two windows or an uncertainty-

adjusted material decline; a serious high-risk category error can require immediate action independent of the aggregate threshold. **[PROPOSED]**

For binned features with fixed edges, calculate **Population Stability Index (PSI)** using smoothed proportions; provisional **PSI ≥ 0.10** triggers review and **PSI ≥ 0.20** triggers investigation. These are **tunable operating heuristics, not universal statistical guarantees**. PSI measures covariate/output distribution shift, not truth-label accuracy. Keep raw-data quality and novelty alerts separate from PSI to avoid false comfort when data has too few labels. **[PROPOSED]**

$$\mathrm{PSI} = \sum_b (p_{t,b} - p_{0,b}) \log\left(\frac{p_{t,b}}{p_{0,b}}\right) \qquad \text{F1 alert}: \; F1_t \le 0.90 F1_0$$

DESIGN REVIEWER — Does an alert mean retraining the model automatically?

ENGINEER — No. The response hierarchy is (i) check ingestion/ schema defects, (ii) seek missing or changed merchant identities and review labels, (iii) revise **approved aliases** if corroborated, (iv) only then train a **challenger** if enough new independent labels exist. Retrain in an approved environment; compare against the frozen champion on untouched merchant-family holdout and minority/high-risk strata; package signed artefacts, record decision owner, canary/parallel-run where permitted, then promote by authorisation with a rollback plan. **Production MLOps is a governance workflow, not scheduled model replacement. [PROPOSED]**

DESIGN REVIEWER — Provide executable-like monitoring logic without implying it runs today.

```
# PROPOSED monthly policy logic, not deployed in Release 2

def model_review_gate(validated_baseline_f1, observed_f1, labelled_ok, psi):
    alerts = []
    if psi >= 0.20:
        alerts.append('INVESTIGATE_SHIFT')
    elif psi >= 0.10:
        alerts.append('DRIFT_WARNING')
    if not labelled_ok or observed_f1 is None:
        alerts.append('INSUFFICIENT_LABELS')
    elif observed_f1 <= 0.90 * validated_baseline_f1:
        alerts.append('F1_DROP_REVIEW_CHALLENGER')
    return alerts or ['RETAIN_APPROVED_CHAMPION']

# No automatic training, promotion or external inference.
```

09 — What is not yet proven, and how to make decisions auditable

DESIGN REVIEWER — List the highest-risk uncertainties.

ENGINEER — We do **not** have independent production precision for approved aliases; merchant/product ground truth for all descriptions; externally validated switches or causal savings; source-authoritative correction sequencing; completed October hidden-file evaluation; Spark/Iceberg parity; representative 15m load/cost results; deployed PSI/F1 monitors; or a live runtime high-risk override. Claiming these are solved would weaken the work. **[UNKNOWN]**

DESIGN REVIEWER — What is the minimum honest first-week production plan?

ENGINEER — Agree the bank's input contract and correction authority; validate transfer/retention boundaries; run synthetic October schema and resend cases; establish manifest/row-count/CDC invariants; port a bounded shard to the approved platform and compare **byte-level and feature-level parity** to the reference implementation; baseline all-in cost and timing; build a small independent merchant holdout and review queue; approve alert thresholds with operations/risk owners; test release and rollback. None of these are represented as accomplished cloud deployments. **[PROPOSED]**

10 — Compact decision register (evidence versus intent)

Decision / claim	State	Evidence anchor / validation limit
Migration-aware canonical ingestion	IMPLEMENTED—REPORTED	`outputs/source_eda.json`; source adapters; original July/Aug CSV + Sep JSONL; re-run code in local Release 2
Bank-scoped key, payload audit, deterministic correction precedence	IMPLEMENTED—REPORTED	Local pipeline; reported replay reconciliation; source authority still UNKNOWN
Four required CSVs and idempotent rerun	TESTED—RECORDED	`outputs/workflow_rerun_verification.json`; 54/54 recorded tests, output SHA checks
Approved aliases + TF-IDF; MiniLM zero vote	IMPLEMENTED—REPORTED	`categorise/core.py`, `classification/curated_aliases.csv`, `outputs/pr_evaluation/`
0.789 top-1, 0.635 macro-F1 (exploratory)	TESTED—RECORDED	57-label leave-one-description-out benchmark; NOT independently held-out precision

Decision / claim	State	Evidence anchor / validation limit
21 descriptions, 15,033 occurrences review-required	OBSERVED / IMPLEMENTED—REPORTED	`scripts/audit_merchant_registry.py`, merchant audit outputs; acceptance != accuracy
High-risk Admiral automatic-acceptance override	PROPOSED	Shadow policy identifies 851 risky occurrences; runtime override NOT implemented
6,681 bills; 209 switches	TESTED—RECORDED candidates	`outputs/bill_validation_summary.json`, `outputs/switch_validation_summary.json`; switches NOT externally confirmed
ApTap AWS S3/Glue/Iceberg architecture	PROPOSED	Security/contracts, Spark parity, capacity and cost need validation
Fully bank-contained approved offline deployment	PROPOSED	Bank product/security approvals and on-prem software catalogue UNKNOWN
Independent benchmark, macro-F1 gate, PSI, champion/challenger	PROPOSED	No adjudicated production baseline; monitoring NOT deployed

11 — Submission provenance and full-history requirement

DESIGN REVIEWER — Can this document replace the AI conversation history requested by ApTap?

ENGINEER — No. It is a **synthetic editorial reconstruction**, even if authored in a genuine new ChatGPT session. It is suitable for explaining the engineering design and documenting decisions, but does not establish what was said in the original chronological chats. The employer requires **complete original conversations, valid share links and export files**, alongside any actually used IDE/coding-agent interactions. Edited summaries, knowledge bases and reconstructions must be labelled *supplemental*, never passed off as original logs. **[REQUIRED submission disclosure]**

Companion evidence: `ApTap_Engineering_Knowledge_Base.md`; `NOTES_ApTap_PartD_Revised.md` (two-page Part D text); the four required CSVs, report, local test logs and real AI transcript exports from the user's project. This document does not assert those files have all been included in the submission archive.

Proposed monthly production handoff (both cloud scenarios)

Stage / owner	Required evidence	Hold or escalation condition
Intake — platform/data engineer	Signed complete manifest; full-file hashes; delivery-ID log; source contract version	Timeout, incomplete chunk, unsupported schema or checksum mismatch → stop promotion and quarantine
Silver — data quality owner	Reconciled raw/accepted/ rejected counts; duplicate and changed-payload audit; booking versus delivery period	Unexplained row deficit, missing identity, invalid sign/date or correction ambiguity → review source evidence
Gold — analytics owner	Candidate bill/switch evidence IDs; changed-window recomputation; report cut-off; risk-policy version	Missing provenance, stale month, unexpected high-risk acceptance → withhold trusted commercial publication
MLOps — model steward	Distinct merchant review load, PSI/novelty, adjudicated macro-F1 and per-class PR if labels permit	PSI warning/investigation, 10% relative F1 decline, or severe high-risk error → investigate; no automatic promotion
Release — bank/ApTap change authority	Signed candidate manifest, dependency inventory/SBOM, regression parity, canary/rollback record	Failed holdout safety check or missing authorised sign-off → retain previous champion

Separation of responsibilities. ApTap can maintain the *design and code* across both scenarios; only scenario A grants ApTap an authorised customer-data processing boundary. In scenario B the bank owns credentials, runtime, telemetry, labels, model approvals and deployment. This distinction applies to monitoring samples as well as raw transactions. **[PROPOSED]**

Footer note for every PDF page: SYNTHETIC ENGINEERING DESIGN RECONSTRUCTION — NOT AN ORIGINAL AI CHAT TRANSCRIPT.